Project Statement: Supercompressible metamaterial design

Miguel A. Bessa (miguel_bessa@brown.edu)[1]

[1]Associate Professor at Brown University, USA

ENGN2350 Data-Driven Design and Analysis of Structures and Materials

Objective: Design a supercompressible metamaterial by applying known or new machine learning and optimization methods in collaboration with AI agents.

General Instructions: This is an **individual** project. Deliver **one ZIP folder** uploaded to CANVAS, containing **one PDF report** of at most **15 pages**[1] (11pt font, 2.5 cm margins, excluding the cover page and the references), and **one notebook per task**. Start by copying all contents in the template folder `3dasm_supercompressible_project_template/` to your folder `your_Project/`. The exact contents are listed in the section *Deliverables* at the end of this project statement.

Grading: every answer must be gradeable from the **report pdf alone** (see *How the project is graded* at the end of this project statement). The notebooks are the source code that contain all the work that generates the results. The notebooks need to run from top to bottom with the saved runs of your agents and produce every result of the report, and they should be easy to read by including proper comments. Please also add an `environment.yml` file inside the .zip for reproducibility.[2]

Code of conduct: You pledge to uphold the Brown academic code in all respects with regard to this Final Project.

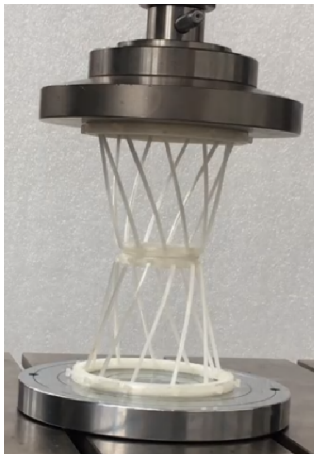
Important notes on collaboration vs. plagiarism:

1. It is acceptable to share progress and discuss ideas with your colleagues, but copying parts of the **report or code** from colleagues or from other students' reports remains plagiarism and will result in a Final Project grade of **zero**.
2. Use of AI coding agents is allowed and encouraged, consistent with the course README. Any AI assistant may help you write your code and your report, but every agent that your code starts (the `adda` agents of Tasks 4, 5 and 6) must use **Claude Haiku**; we check this in the study folders you submit. Your Claude seat has a usage limit: set Claude Code itself to Sonnet or Haiku (`/model`) to make it last. We expect you to be able to explain and defend every part of the code and the report you submit.
3. **Originality** when choosing machine learning and optimization models will be **rewarded**, and outstanding designs earn an A+ (see *How the project is graded*).

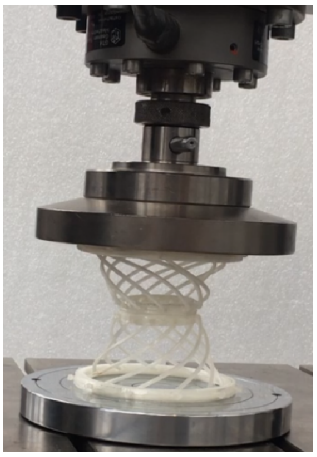
Due date: until 11:59pm of day announced in course home page.

[1]Pages beyond the fifteenth (excluding the cover page and the references) are not read.

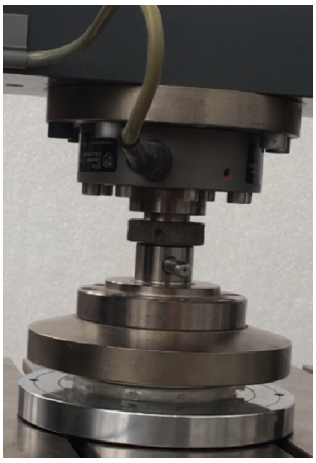
[2]Simply run `mamba env export --from-history > environment.yml` if you used a mamba env, otherwise ask your TA or AI agent how to export it.



(a) Undeformed metamaterial.



(b) 50% deformation.



(c) More than 90% deformation.

Figure 1: 3D printed super-compressible metamaterial designed with machine learning [1].

Introduction

For centuries creating new materials and structures followed an experimental trial-and-error design process. However, a new paradigm has emerged where agentic AI together with machine learning and/or optimization methods are used to accelerate the design process, sometimes leading to unprecedented performance. This project will illustrate that even simple methods can lead to an interesting discovery of a 3D printed metamaterial that, despite being fabricated with fragile polymers (elongation at break around 4%), it exhibits reversible supercompressibility ($> 90\%$ effective compressive strain[3]).

Figure 1 shows the metamaterial that was designed without doing any experiment before testing the successful design. Instead, the design was found by a computational data-driven approach [1] where experiments were used for validation, not discovery. With brittle polymers, that article fabricated two designs: a macroscale design that reached recoverable effective stresses around 0.1 kPa at effective strains beyond 94%, and a microscale design that reached recoverable effective stresses beyond 100 kPa at effective strains around 80%. In theory, fabrication with advanced materials could lead to much higher effective stresses.

The Python framework: f3dasm

You will use the framework for data-driven design and analysis of structures and materials (`f3dasm`) as a systematic approach to inverting the metamaterial design process [2, 3]. The framework integrates:

- **design of experiments**: in this work, it consists of sampling the input variables that describe the geometry, properties and external conditions of the system to be evaluated. This determines the search space;
- **data generation**: here, by performing computer simulations of the metamaterial by the finite element method, resulting in the creation of a response database;
- **machine learning**: here, by creating a surrogate model, trained on the response database;
- **optimization**: here, by conducting optimization on the surrogate model to find an optimum design. If the surrogate model is accurate, the predicted optimum design will be similar to the optimum obtained directly from the simulations (and, eventually, the design that is fabricated will agree with the simulations).

In this assignment, you will only focus on the machine learning and optimization modules of the framework. In other words, the data generation and the design of experiments modules were already done beforehand.[4]

Note that `f3dasm` is a framework. Optimizers and machine learning models may be called directly from their own packages (e.g. `scipy`, `scikit-learn`, `optuna`). You will also use the `adda` package to set up the agentic workflows, when needed.

[3] The effective compressive strain is the reduction in height of the metamaterial divided by its initial height.

[4] The data generation as well as the design of experiments were done using f3dasm, as presented in the Lectures. The data generation consists of running a commercial finite element code (Abaqus) for every input sampling point determined by a design of experiments method.

Aim of the project

This project focuses on a hands-on approach to understand how to use machine learning and optimization methods to design this metamaterial using a data-driven framework. At the end of this project, you will gain:

- Introductory knowledge on design of experiments: uniform sampling vs. Sobol sequences.
- Basic knowledge on machine learning: distinction between supervised regression and classification, simple dataset preprocessing, and selection of an appropriate algorithm.
- Elementary knowledge on optimization
- Practical knowledge on extrapolation: how the way you represent your inputs, and the model you choose, decide whether a surrogate can be trusted beyond the data it was trained on.
- Working with a team of AI agents on an engineering study: deciding every choice before prompting and checking the agents' results independently; and then letting a team of agents decide by itself, understanding what it did, and comparing it with your own decisions.

Provided data and resources

We will provide two datasets that you should use to explore the design space and come up with creative solutions:

- A dataset with 1000 experiments that is parametrized by 7 parameters (`supercompressible_7d`).
- A dataset with 1000 experiments that is parametrized by 3 parameters (`supercompressible_3d`).

As previously mentioned, the data was already created with the `f3dasm` package and it is shared with you in the course GitHub repository (inside `3dasm_supercompressible_project_template/data`). You will not be able to generate additional data points.

Design-of-experiments

The supercompressible metamaterial is parameterized by 5 geometric parameters and 2 material parameters [1]. The geometry is defined by the top and bottom diameters, D_1 and D_2, the height P and the cross-section parameters of the vertical longerons: the cross-sectional area A, moments of inertia I_x and I_y, and torsional constant J, see Figure 2. The isotropic material is defined by its elastic constants: Young's modulus E and shear modulus G.

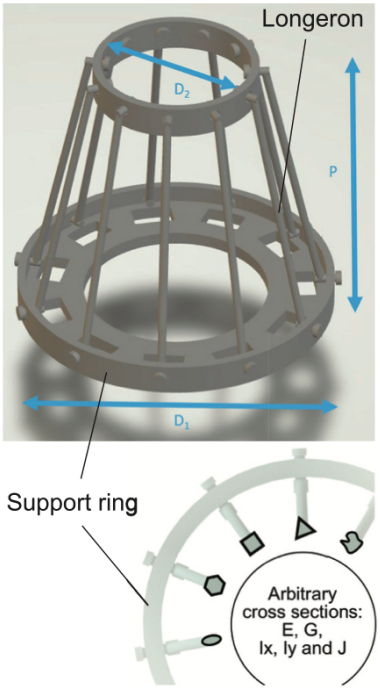


Figure 2: Supercompressible metamaterial building block with generalized cross-section for the longerons (x is the radial direction; y is the tangential direction) [1].

Due to the principle of superposition both the geometric and material parameters can be scaled by one of its dimensions/properties (here D_1 and E). Therefore, the variables that you will find in the dataset are:

$$\text{Design of Experiments 1:} \quad \frac{D_1 - D_2}{D_1}, \frac{P}{D_1}, \frac{I_x}{D_1^4}, \frac{I_y}{D_1^4}, \frac{J}{D_1^4}, \frac{A}{D_1^2}, \frac{G}{E}$$

This is a 7-dimensional problem and learning the response surface may require a significant amount of training points[5]. Therefore, you will also consider a simpler version of the problem in 3 dimensions, defined by constraining the longerons' cross-section to be circular with diameter d, and choosing a particular material ($G/E = 0.3677$), leading to the following 3 features:

$$\text{Design of Experiments 2:} \quad \frac{d}{D_1}, \frac{D_1 - D_2}{D_1}, \frac{P}{D_1}$$

The parameter names and the bounds of the sampling domain of each design variable can be found in the `domain.pkl` *pickle* files. These bounds can be wider than the range of the data: the range of the data is the minimum and maximum of each input in `input.csv`.

[5] Remember the "curse of dimensionality"!

Input data

Using a Sobol sequence[6] we have sampled the design space. Table 1 and 2 show the names of the parameters for the 7D and 3D dataset respectively.

expression	parameter name
$\frac{D_1-D_2}{D_1}$	`ratio_top_diameter`
$\frac{P}{D_1}$	`ratio_pitch`
$\frac{I_x}{D_1^4}$	`ratio_Ixx`
$\frac{I_y}{D_1^4}$	`ratio_Iyy`
$\frac{J}{D_1^4}$	`ratio_J`
$\frac{A}{D_1^2}$	`ratio_area`
$\frac{G}{E}$	`ratio_shear_modulus`

Table 1: Input variables in the 7-dimensional dataset: `input.csv`

expression	parameter name
$\frac{D_1-D_2}{D_1}$	`ratio_top_diameter`
$\frac{P}{D_1}$	`ratio_pitch`
$\frac{d}{D_1}$	`ratio_d`

Table 2: Input variables in the 3-dimensional dataset: `input.csv`

The samples and number of points considered can be found in the `input.csv` file.

Output data

For each data point (i.e. for each material design) we can use nonlinear finite element analyses (RIKS arc-length method) to predict the complete buckling and post-buckling behavior. From the analyses, we can understand if a material is coilable[7] and compute the critical buckling stress[8] σ_{crit} and the energy absorbed E_{abs}. The datasets have information about these three outputs.

For the 7-dimensional problem, the coilability of a material is simply defined by a Boolean variable, i.e. the material can either be or not be coilable. For the 3-dimensional problem, we can extract more information and understand also if within the coilable materials there is yielding (plasticity) or if they are reversible (elastic).

Concerning the output quantities (targets), the critical buckling stress σ_{crit} is deterministic and it is directly provided by the finite element analyses, while the energy absorbed E_{abs} is a random variable (geometric imperfections influence absorbed energy), and is computed from the stress-displacement curve (area under the curve) provided by the finite element analyses. The strategy to compute this variable is fairly robust (we interpolate the stress-strain curve using a piecewise cubic Hermite interpolating polynomial and then compute the integral by applying Simpson's rule), but it is still prone to errors.

Each nonlinear simulation starts from a geometry perturbed by its first buckling mode, as a manufactured specimen would be, scaled so that the largest nodal rotation of the perturbation is a random amplitude a (in radians). The amplitude is lognormal, with $\ln a$ normally distributed with mean -2.705 and standard deviation 0.294 (a mean of about $4.0°$ and a standard deviation of about $1.2°$). The amplitude a used in each simulation is given in the data as `imperfection`. It is not a design variable. You cannot choose it, and every new specimen of the same design gets a new value. The energy absorbed and, for the 3-dimensional problem, whether a coilable design yields depend on it, while σ_{crit} does not. It is missing (`NaN`) for the designs that are not coilable, which were not simulated beyond buckling.

[6] An unscrambled Sobol sequence. The 3-d inputs are its first 1000 points, which the Sobol sampler of `f3dasm` reproduces; the 7-d inputs are 1000 points of a longer Sobol sequence over the 7-d domain, all with $G/E \leq 0.159$ (see Task 5).

[7] A design is coilable when, under axial compression, its longerons collapse by coiling: the first buckling mode rotates the top ring about the vertical axis without moving it sideways. This is what allows the metamaterial to be compressed almost flat.

[8] Critical buckling stress is defined as the critical buckling load divided by the area of the bottom ring of the metamaterial.

Due to unsuccessful simulations, there are missing values of the energy in the datasets (they are stored as `NaN` in the data). You can choose to ignore them or learn about strategies to complete the missing data (missing data is very common in real problems like these). Table 3 shows the names of the parameters for both of the datasets.

expression	**parameter name**
coilability	`coilable`
σ_{crit}	`sigma_crit`
E_{abs}	`energy`
imperfection amplitude a (rad)	`imperfection`

Table 3: Output variables in the datasets

The critical buckling stress `sigma_crit` is given in kPa and the absorbed energy `energy` in kJ/m^3.

The values of the `coilable` parameter are encoded in the following way

- 0 = not coilable
- 1 = coilable
- 2 = coilable but yields (i.e. the material would not be reversible; and so, this is still not a good design)

A coilable design yields when the largest strain in its longerons exceeds 2% once it is fully coiled. Some simulations stopped before the design coiled fully. For these, that strain was predicted from the simulations that did, for the same design inputs and imperfection amplitude.

The post-processed output quantities of the simulations can be found in the `output.csv` file.

In summary, we provide you with two datasets, each containing 3 output variables (coilability, critical buckling stress σ_{crit} and energy absorbed E_{abs}) and the imperfection amplitude of each simulation: i) 7-dimensional problem (generalized longeron's cross section and different elastic material properties) and ii) 3-dimensional problem (circular longeron's cross section and fixed material properties). With this information you will design your own metamaterial after finding the machine learning models to classify and regress the design space (creating what is called a "surrogate model"), followed by optimization on the surrogate model. Note that you can optimize the metamaterial with different goals in mind, e.g. reversibly coilable metamaterial with highest energy absorption capability or maximum buckling strength. In order to get there, you will start by answering some questions.

Getting started

Installation

1. Copy the contents of `3dasm_supercompressible_project_template/` into `your_Project/` (both of these folders are found in the `Projects` folder of the course's GitHub repository). The template holds the two datasets (`data/`), one skeleton notebook per task (`task1.ipynb` to `task6.ipynb`), a report template (`report_template/`), `DELIVERABLES.md` and `check_submission.py`. Do all your work in `your_Project/`, which `git` ignores: it becomes your submission (see *Deliverables*). Never edit the template itself, so that `git pull` can update it.

2. In your `3dasm` environment (Python 3.11 or 3.12), install the `f3dasm` package and its agentic layer, `adda`, which you will use in Tasks 4, 5 and 6. As in Lecture 19, `adda` is installed from a wheel. Download `adda-0.2.0-course-wheel.zip` from this Google Drive link, unzip it, and run from the unzipped folder

   ```
   uv pip install "f3dasm[scipy]==2.4.0" pypdf
   uv pip install "./adda_f3dasm-0.2.0-py3-none-any.whl[viewer]"
   ```

   Without `uv`, use `pip install` instead. (`pypdf` lets `check_submission.py` read your report.) The file `INSTALL.txt` in the zip gives the checksum of the wheel and how to check the installed version.

3. If you work with Claude Code, put the line `@DELIVERABLES.md` in the instructions file (`CLAUDE.md`) of `your_Project/`, so that your agent reads the rules of the deliverables from the start. The rest of the instructions file is yours to write.

Questions to be answered

Data characterization

1. Although commonly underestimated, an important step in data analysis is the preprocessing step. Therefore, **before** considering to use any machine learning model, do at least the following:

   1.1. For both datasets (3-d and 7-d) indicate the range of each input variable (its minimum and maximum in `input.csv`) and the number of available points for each output variable.

   1.2. For the 3-d dataset plot all the points in 2-d scatter plots for all possible combinations of the input features, i.e. every pair of features as x and y of the scatter plots. Observe the sampling points and how they are distributed in the domain. Report your conclusions about the design of experiments strategy, but only show in the report the scatter plots for the first 100 points of the database (no need to plot every point in the database). Also include a 3-d scatter plot of the first 100 points (both plots should convey the same information). What can you conclude about the characteristics of the sampling method used (Sobol sequence)?

   1.3. For both datasets, create histograms of each continuous output, and a table of the number of points in each category of `coilable`. What do they tell you about how to model these outputs in Task 2?

   1.4. For the 3-d dataset, find the point corresponding to a material that is **reversibly coilable** (without yielding) that has maximum critical buckling stress, and another point that has maximum energy absorption capability (also for a reversibly coilable material). Report the feature values and every output value for **both** points.

Note that up to this stage you have not done any machine learning or optimization. Yet, the information you collected up to now is useful to establish a baseline for our next investigation.

Finding good machine learning models

2. Start your machine learning investigation by focusing on training different machine learning algorithms and evaluating which ones are better suited for this problem. Consider the typical split into training and testing data of 75% of data for training. Split the rows of each dataset once, with the seed 123 (for example `train_test_split(..., test_size=0.25, random_state=123)`), and use this split for every model of Tasks 2 and 3, keeping for each target only the rows where it is available. Preprocessing the inputs (for example scaling them) is allowed for every model, including the ones asked for with default hyper-parameters; report it. You will do the following tasks for both datasets, but start with the 3-d dataset. Please create meaningful representations (plots) of the predictions, i.e. you should not include every plot you make during your investigation (just the key ones). You need to keep your report clean and concise, otherwise you will not be able to fulfill the page limit.

   2.1. Tasks to do for the 3-d dataset:

      2.1.1. For the **classification** problem:

         A. Fit as many classifiers as you like, including at least the C-Support Vector Classifier from `scikit-learn` with the default hyper-parameters. Report how many classifiers you trained, and the worst, the median and the best of them by testing accuracy.

         B. Explain in your own words why the worst, the median and the best classifiers performed as they did, and select the best classifier.

         C. Repeat the two steps above, but now considering only two categories (*coilable*[9] and *not coilable*).

      2.1.2. For the **regression problems** (i.e. prediction of the critical buckling load and the energy absorbed):

         A. Fit as many regression models as you like, including at least the Gaussian process regressor from `scikit-learn` with the Matern kernel and the smoothness parameter $\nu = 2.5$. Report how many regression models you trained, and the worst, the median and the best of them by testing mean squared error.

[9] *Coilable (but yields)* must be considered as *coilable*

B. Explain in your own words why the worst, the median and the best regression models performed as they did, and select the best regression model.

C. Compare the best solutions found for each problem. Report the adequacy of each algorithm for noisy/noiseless problems.

2.2. Repeat the same steps (2.1.1 and 2.1.2) for the **7-dimensional problem**. Its data has only two categories (coilable and not coilable), so its classification is done once.

Note: For all the models that you report, you must report the hyperparameters, the error metrics, and the number of training and testing points used[10]; list every model you trained, with its error metrics, in the file `task2_models.csv`. When the test errors of several models differ by less than your test set can resolve, say so, and choose between them with a reason. Don't be afraid to report the results of models that perform poorly: the knowledge gained from failed solutions is as valuable as the one gained from solutions that actually work. In some cases, it may not even be feasible to get a meaningful prediction with a particular algorithm (in that case, just report that no fit was possible to obtain with that algorithm). For **classification problems** you must always report *accuracy score* and, whenever possible, *precision score*, *recall score* and *F1 score*. You may consider creating a **confusion matrix** and interpreting it. For **regression problems** you must always report R^2 *score* and *mean squared error*. You should also provide a short description of a new machine learning algorithm considered in your project that was not covered in class.

Evaluating the best machine learning models you found

3. Machine Learning algorithms have several hyper-parameters and the quality of the predictions depends on the size of the training data. Understanding how the hyper-parameters and the data used affect the model prediction ability is important. Therefore, for the **3-dimensional** problem, using the same training and testing data as in Task 2:

3.1. Investigate the **C-Support Vector classifier** and the **Gaussian process regressor** algorithm to:

3.1.1. Study the influence of the **number of training points** in the performance of the algorithm, taking subsets of your training data and keeping the testing data fixed[11].

3.1.2. Study the influence of the following hyper-parameters:

- for the C-Support Vector classifier the value of C
- for the GPR the smoothness parameter (ν) of the Matern kernel

3.1.3. Perform hyper-parameter optimization for the above mentioned hyper-parameters and the other hyper-parameters of these two models that you consider relevant. Choose as your objective the accuracy (classification) and the mean squared error (regression) estimated by cross-validation on the training data, so that the testing data stays unseen; report the testing error of the optimized models at the end. Consider **Random Search**, **TPESampler**[12] and **at least one other different optimization algorithm** and report the objective with respect to the number of hyperparameter optimization iterations for each of the runs.

3.2. Repeat 3.1.3 for your best classifier and your best regressor of 2.1.1 and 2.1.2 (if one of them is the C-Support Vector classifier or the Gaussian process regressor, say so and use your result of 3.1.3). These optimized models are the ones you use in 4.4.

Optimization

4. Commonly, the ultimate goal of having a description of the design space is to find the best solution for a given objective and under some constraints. Comparing many optimizers fairly is tedious, as each optimizer has to be run several times from different starting points, on more than one objective and more than one surrogate, and every run has to be recorded in the same way. This is the kind of work that AI agents can take off your hands, as long as you tell them precisely what to do; automating it is standard practice in engineering. In this task you will set up the comparison as a **campaign** run by a team of agents

[10] Use tables whenever possible.

[11] You may not have yet gained intuition about the importance of the number of data points, but imagine that to collect the dataset you need to spend 1 minute to simulate each data point. A dataset with 50000 data points would need about 35 days of CPU-time to generate your data.

[12] For example `optuna`, or the `cmaes` package.

you set up with `adda`[13]. Your prompts must give the agents implementation tasks only: every choice and every conclusion must be yours, and must appear in your tables of decisions and in your report. We grade this from your prompts and problem statements. Consider the following tasks for the **3-dimensional problem** only:

4.1. Before considering any optimization algorithm, report a search[14] for the optimum points which have been classified by the C-Support Vector classifier as reversibly coilable (without yielding) that satisfy the following:

4.1.1. Higher critical buckling stress.

4.1.2. Higher energy absorption capability.

4.2. Write the problem statement of an `adda` study that compares the **Nelder-Mead optimizer**, the **CMAES optimizer** and **2 other optimization algorithms of your choice** on the Gaussian process regressor paired with the C-Support Vector classifier (both as required in Task 2), to determine in one campaign the reversibly coilable point with higher critical buckling stress and the reversibly coilable point with higher energy absorption capability. Before writing any prompt, decide and report in a table: the two optimizers you added and why, how the constraint is imposed, the number of repetitions and starting points of each optimizer, the evaluation budget, the seeds, and what you record for every run. You are welcome to divide the implementation into whatever agentic workflow you find efficient. The campaign runs from `task4.ipynb`. Run it once, in the study folder `studies/task4_2`[15]. Report the objective with respect to the number of iterations for each optimizer, over all its repetitions.

4.3. Compare the solutions obtained in 1.4, 4.1 and 4.2. Check at least one result of the campaign independently of the agents. Report if you needed to correct the agents, adapt the prompts, or adapt the agentic workflow to generate the final results that you are presenting.

4.4. Repeat 4.2 but now with your best-performing classifier and regressor in combination with the optimal hyper-parameters acquired in 3.2, or with a classifier you justify as better suited to optimization, by changing your problem statement rather than writing a new campaign. Run it once, in `studies/task4_4`. We will simulate your designs, so you may refit your final models on all 1000 points of the 3-d dataset. Submit the two designs you choose from the results of this campaign in the file `designs_3d.csv`: a column `design` (`sigma_crit` for the design of higher critical buckling stress, `energy` for the design of higher energy absorption) and the three inputs of the 3-d dataset, each within the range you reported in 1.1. We will simulate them. Whether a design yields, and how much energy it absorbs, depend on small random imperfections of the geometry[16], so we will simulate each design 20 times, each time with a new random imperfection. The result of each design is the median of its 20 simulations (critical buckling stress or absorbed energy), where a simulation in which the design is not coilable or yields counts as zero. In other words, we expect you to predict the median of a noisy (aleatoric) random variable. Estimate the probability that each of your designs stays reversibly coilable, and compare it with the designs of the data nearest to it.

Optimization considering new material (extrapolation)

5. In practice, the data available to a designer might not cover every design they are allowed to build. Every 7-d experiment you received was simulated with a material whose shear-to-Young's modulus ratio is $G/E \leq 0.159$. A new polymer now makes the full range of the 7-d domain available, up to $G/E = 0.45$. Every other input of your designs must stay within the range of the 7-d dataset that you reported in 1.1 (the minimum and maximum of each column of `input.csv`). Consider the following tasks for the **7-dimensional problem** only:

[13] Every agent must use the Claude Haiku model (see the notes on collaboration). The `adda` documentation explains how to write a problem statement and how to define your own agents.

[14] In other words: use the C-Support Vector classifier to select the points in the 3-d dataset that are reversibly coilable, and then find the maximum value for the corresponding output within the list of points that are reversibly coilable (maximum of a list does not need an optimization algorithm).

[15] You may run a campaign more than once while you develop its problem statement; submit the run you report, and say in 4.3 what you changed and why. This applies to every campaign of Tasks 4 and 5.

[16] The imperfection amplitude a is random (see the section *Output data*). Each design in your data was simulated once, with the amplitude given in `imperfection`.

5.1. Run an `adda` campaign, as in 4.2, in `studies/task5_1`, that compares preprocessing options, classifiers and regressors for predicting designs with $G/E > 0.159$, where the 7-d dataset has no points[17]. Before writing any prompt, decide and report: the preprocessing options, classifiers and regressors that are compared (starting from your choices in Tasks 2 and 3), how each combination is assessed for $G/E > 0.159$, and what has to be recorded. The agents carry out the comparison; report the conclusions you draw from it, which are yours.

5.2. Choose yourself **five** designs that you expect to have the highest critical buckling stress while being coilable. We will simulate your designs, so for this choice you may retrain your final models on all 1000 points of the 7-d dataset. Submit a file `submission.csv` with one row for each of your five designs, containing its seven inputs (named as in the 7-d dataset) and your predicted σ_{crit} in kPa, in a column `predicted_sigma_crit`. A design that is not coilable or is outside these ranges scores zero, while a coilable design scores its σ_{crit}. You are allowed to have duplicated rows, and your final score is the median of your five simulated designs.

5.3. Compare your designs with the best coilable design in the data you received. Discuss how far your designs are from that data, and try to estimate the epistemic uncertainty of your predictions.

Letting the agents decide

6. In Tasks 4 and 5 you made every choice and the agents implemented it. Now do the opposite: design one agentic workflow with `adda` that solves the two design problems by itself, and compare what it achieves with what you achieved. As usual, every agent must use Claude Haiku. Remember that your Claude seat has a usage limit, so plan the budget of the run.

6.1. Write the problem statement of one `adda` study that delivers the two 3-d designs of 4.4 and the five 7-d designs of 5.2, under the same rules and scoring. The problem statement gives the agents the goal, the data, the rules, the scoring and the budget, and asks them to record every strategy they consider and how they assess it. It must not name methods, nor tell the agents anything you found in Tasks 1 to 5: the agents decide. The team is yours to design: how many agents, their roles (for example a literature reviewer, an analyst, a critic) and how they work together. Report the team in a diagram or a table and justify it. Run the study once[18], in `studies/task6`. `task6.ipynb` writes the designs of the agents to `designs_3d_agents.csv` and `submission_agents.csv`, in the formats of `designs_3d.csv` and `submission.csv`. We will simulate them as we simulate yours.

6.2. Report in a table the strategies the agents considered, and the worst, the median and the best of them by the agents' own assessment. Explain the best strategy as an algorithm, in your own words, and say whether you trust the way the agents assessed their strategies.

6.3. Compare the designs of the agents with yours, for each problem: where they agree, where they differ, and which designs you expect to perform better, and why. Your grade does not depend on which designs win.

[17] A test error cannot be computed where there is no data. Think about what else could tell you whether the predictions of a model are sensible there.

[18] You may run it more than once while you develop the workflow; submit and report the run you keep.

How the project is graded

You will be graded based on your **report**. An AI model will read your notebooks and check that the work the report describes is the work the notebooks did; the notebooks are not graded on their own, and an answer that is only in a notebook is not graded. Write every relevant answer in the report so that it can be graded from the report alone. The project receives one of four grades:

- **A+ (100%)**: An outstanding report whose designs obtained from Tasks 4.4 and 5.2 perform remarkably well when evaluated in our finite element simulations.
- **A (90%)**: every task answered, correctly and in your own words, with the choices justified.
- **B (75%)**: answers that are incomplete or contain errors, or a report that is hard to follow.
- **C (60%)**: a report that shows little understanding of what was done.
- **D (0%)**: project is not delivered. If you deliver something with an honest attempt at solving the project you get 60% for the project.

The designs of the agents (Task 6) are simulated too, but your grade does not depend on them, only on how well you understand and compare them.

Deliverables

Given the class size, there will be no presentations. Submit **one ZIP-file** of your folder `your_Project/` containing the files below; nothing else is graded. The files of the template that are not listed (`check_submission.py`, `DELIVERABLES.md`, `report_template/`) and your `CLAUDE.md` may stay in the folder. `DELIVERABLES.md` summarizes this section for your agents, and this section is authoritative.

- `report.pdf`: a cover page with the title, the course name, your name, your student ID and the date, followed by at most **15 pages** (11pt font, 2.5 cm margins), excluding the references; an appendix, if you have one, counts towards the 15 pages. Show results in tables and figures, and use the text to explain them; make each figure at the size it is printed, so that its labels can be read. For convenience, `report_template/report.tex` is a LaTeX template with this structure and the tables that every report needs; you do not need to use LaTeX, but keep the same structure. Start the answer to each question with its number in bold, **Answer 1.1.**, **Answer 1.2.**, and so on, in the order of this statement, so that the graders find every answer. Every number and figure of the report names the notebook and section it comes from, for example (task4, 4.4).
- `task1.ipynb` to `task6.ipynb`: one notebook per task, each started from its skeleton.
- `task2_models.csv`, `designs_3d.csv`, `submission.csv`, `designs_3d_agents.csv` and `submission_agents.csv`, written by `task2.ipynb`, `task4.ipynb`, `task5.ipynb` and `task6.ipynb` (the last two) with the cells provided in the skeletons.
- `studies/`: one `adda` study folder per campaign, named `task4_2`, `task4_4`, `task5_1` and `task6`, each holding the run you report.
- `environment.yml`, `data/` as released, and `cache/` (written by the notebooks, at most 50 MB).

The notebooks must follow these rules, because the graders read them and nothing else of your code:

1. **Structure.** Keep the first cell and every heading of the skeleton (one per subtask, numbered as in this statement), and add as many cells as you need under each heading. Close each section with its **Answer** cell, which states your answer in your own words.
2. **Self-contained.** A notebook imports only installed packages, never `.py` files of your own, and reads only `data/`, `cache/` and `studies/`. If Task 4 needs the models of Tasks 2 and 3, `task4.ipynb` fits them again.
3. **Saved runs.** As in the lectures, a notebook shows work that was done once and saved. The cell that starts an `adda` campaign runs only if `RUN_AGENTS = True`; otherwise the notebook reads the records of the study folder. Next to each campaign, the notebook shows its problem statement, its results, and that every agent used Claude Haiku (from the run records). We do not rerun your agents: a new run could give a different result, as a colleague redoing your work could, and that is fine. Expensive computations go through the function `cached` of the first cell, which saves them in `cache/`; with `RECOMPUTE = False`, each notebook runs from top to bottom in under 10 minutes.

4. **Submitted executed**, from top to bottom in a fresh kernel, with `RUN_AGENTS = False` and `RECOMPUTE = False` and all outputs saved.

`check_submission.py` is the first thing the TAs run on your submission. It checks the files, the structure of the notebooks, the CSV files, the study folders and the answer labels of the report, and runs every notebook again. We give it to you so that you are not penalized for a missing file or a broken notebook: run `python check_submission.py` in `your_Project/` before you submit, and fix everything it reports. A submission that fails it is returned to you, once, to be fixed within 48 hours.

References

[1] M.A. Bessa, P. Glowacki, and M. Houlder. Bayesian machine learning in metamaterial design: Fragile becomes supercompressible. *Adv. Mater.*, 0(0):1904845–, October 2019.

[2] M.A. Bessa, R. Bostanabad, Z. Liu, A. Hu, Daniel W. Apley, C. Brinson, W. Chen, and Wing Kam Liu. A framework for data-driven analysis of materials under uncertainty: Countering the curse of dimensionality. *Computer Methods in Applied Mechanics and Engineering*, 320:633 – 667, 2017.

[3] M. P. van der Schelling, B. P. Ferreira, and M. A. Bessa. f3dasm: Framework for data-driven design and analysis of structures and materials. *Journal of Open Source Software*, 9(100):6912, 2024.